Contents

Glassdoor Reviews — Module 3 Lab Guide 2

The Scenario — Your Mission 2
- Where you work . . . 2
- The problem . . . 2
- Your manager's request . . . 3
- Your team's job for the next 2 weeks (Module 3) . . . 3
- After Module 3 . . . 3

How This Guide Works (Read This First!) 3
- We do NOT give you the full code . . . 3

Visualizations — Two Modes 4
- 1. Exploratory charts (for YOU) . . . 4
- 2. Explanatory charts (for LINDA) . . . 4
- Your plotting toolkit . . . 4

Before You Start — Google Colab Setup 4
- Step 1 — Open a new Colab notebook . . . 4
- Step 2 — Connect to Google Drive . . . 4
- Step 3 — Create a project folder . . . 4
- Step 4 — Get the data . . . 4
- Step 5 — Sample 100,000 rows . . . 5
- Colab tips . . . 5

Class 1 — Data Cleaning 5
- Your goal . . . 5
- Inputs . . . 5
- Outputs . . . 5
- Phase A — Explore the data first (15 min) . . . 5
- Phase B — Clean the reviews table (45 min) . . . 6
- Phase C — Make ONE chart for Linda (15 min) . . . 7
- Common mistakes in Class 1 . . . 7
- Self-check before Class 2 . . . 7

Class 2 — Encoding and Scaling 7
- Your goal . . . 8
- Inputs . . . 8
- Outputs . . . 8
- Phase A — Explore the data first . . . 8
- Phase B — Encode and scale . . . 8
- Phase C — Make ONE chart for Linda . . . 9
- Common mistakes in Class 2 . . . 9
- Self-check before Class 3 . . . 9

Class 3 — Feature Engineering 10
- Your goal . . . 10
- Inputs . . . 10
- Outputs . . . 10
- Phase A — Explore the data first . . . 10
- Phase B — Engineer the features . . . 10
- Phase C — Make ONE chart for Linda . . . 11
- Common mistakes in Class 3 . . . 11
- Self-check before Class 4 . . . 12

Class 4 — Feature Selection 12

Your goal . . . 12
Inputs . . . 12
Outputs . . . 12
Phase A — Explore the data first . . . 12
Phase B — Select features . . . 12
Phase C — Make ONE chart for Linda . . . 13
Common mistakes in Class 4 . . . 13
Self-check before Class 5 . . . 13

Class 5 — Pipelines **13**
Your goal . . . 14
Inputs . . . 14
Outputs . . . 14
Phase A — Explore the data first . . . 14
Phase B — Build the pipeline . . . 14
Phase C — Make ONE chart for Linda . . . 15
Common mistakes in Class 5 . . . 15
Self-check before Class 6 . . . 16

Class 6 — End-to-End Lab (THE BIG ONE) **16**
Your goal . . . 16
Time . . . 16
Outputs . . . 16
Phase A — Explore the data first (10 min) . . . 16
Phase B — Build the final dataset (70 min) . . . 16
Phase C — Findings report for Linda (10 min) . . . 17
Grading rubric (100 points) . . . 18
Submit . . . 18

Tips for the whole module **18**

Glassdoor Reviews — Module 3 Lab Guide

Scenario: HR & jobs. Predict if an employee would recommend their company. **Module:** 3 (Data Preparation and Feature Engineering) **Audience:** Adult learners, A2 English. New to AI. **Tools:** Google Colab (no install needed).

The Scenario — Your Mission

Where you work

You and your team are new data analysts at **Glassdoor**. Glassdoor is a website where workers anonymously review their company. They give 1 to 5 stars + write what is good (pros) and bad (cons).

Every month: - **600,000 reviews** are submitted globally. - Some workers click “Yes, I recommend this company”. - Some click “No”. - **Many workers leave the recommend field empty.**

The problem

Glassdoor sells “Employer Insights” reports to corporate HR teams. The reports show: “Your employees rate you 3.4 stars. 65% would recommend you.”

But when 30% of workers leave the recommend field empty, the report is incomplete. HR teams are not happy.

Linda is worried. She calls your team into a meeting.

Your manager's request

Your manager, **Linda** (Director of Insights), tells you:

> "When a worker leaves the recommend field empty, we are blind. We sell incomplete reports.
>
> I need a model. Read the pros, cons, and advice-to-management text. Predict whether the worker would recommend — even when they did not click the button.
>
> Now we deliver a COMPLETE report to every HR team. They pay more for complete reports."

Your team's job for the next 2 weeks (Module 3)

Linda sends you a CSV with **600,000 reviews**. You will sample **100,000** rows to keep it tractable.

Your job in Module 3: > **Turn this CSV into ONE clean file. The clean file will be used to train the model in Module 4.**

The clean file is called `glassdoor_reviews_clean.parquet`. It must have **21 specific columns** (we will see them in Class 6).

After Module 3

Module	What happens
Module 4	Train the recommend model. Linda gets her "predicted recommend" tool.
Module 5	Group companies by review profile. "Pay good, culture bad" vs "Pay low, growth high".
Module 7	Use all 4 text columns. Find aspect sentiment: pay, culture, growth, management.

You use the **same Glassdoor dataset** until the end of Module 7.

How This Guide Works (Read This First!)

We do NOT give you the full code

In this guide you will see:

1. WHAT / WHY / EXPECTED — explained in words

For every step: - **WHAT** you have to do - **WHY** you do it - **HINTS** — function names + arguments - **EXPECTED** — the result you should see

2. Sometimes a code skeleton with blanks

You fill in the blanks (___).

Why no full code?

You will NOT learn if you copy-paste. **Rule:** Redo every copy-pasted step by yourself.

Visualizations — Two Modes

1. Exploratory charts (for YOU)

Fast, ugly. Goal: "What does this data look like?"

2. Explanatory charts (for LINDA)

Clean, labeled, one clear message. Goal: "Linda, look at this."

Your plotting toolkit

Plot	When to use it	Function name
Histogram	Shape of a numeric column	`plt.hist()`
Bar chart	Count of each category	`df['col'].value_counts().plot.bar()`
Box plot	Outliers in a numeric column	`sns.boxplot()`
Heatmap	Correlation between many columns	`sns.heatmap(df.corr())`

Before you start: `import matplotlib.pyplot as plt` and `import seaborn as sns`.

Before You Start — Google Colab Setup

Step 1 — Open a new Colab notebook

1. Go to https://colab.research.google.com
2. Sign in with your Google account.
3. Click **"New notebook"**.
4. Name it `glassdoor_module_3.ipynb`.

Step 2 — Connect to Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Step 3 — Create a project folder

```
import os
os.makedirs('/content/drive/MyDrive/glassdoor_lab', exist_ok=True)
%cd /content/drive/MyDrive/glassdoor_lab
```

Step 4 — Get the data

Option A — Direct from Kaggle:

1. Free Kaggle account.
2. Upload `kaggle.json`:

```
from google.colab import files
files.upload()
```

3. Then:

```
!mkdir -p ~/.kaggle && mv kaggle.json ~/.kaggle/ && chmod 600 ~/.kaggle/kaggle.json
!pip install kaggle -q
!kaggle datasets download -d davidgauthier/glassdoor-job-reviews
!unzip -q glassdoor-job-reviews.zip -d data
!ls data/
```

Option B — Upload by hand in Colab's file panel.

Step 5 — Sample 100,000 rows

The full dataset is 600,000 rows. Sample 100,000 to keep Colab fast:

```
import pandas as pd
df = pd.read_csv('data/glassdoor_reviews.csv').sample(100_000, random_state=42)
print(df.shape)
```

Colab tips

Tip	Why
Save your notebook to Drive often	Colab disconnects after 12 hours.
Save intermediate `.parquet` files to Drive	Files in `/content/` disappear when Colab disconnects.
Share the notebook with teammates (Share button, top right)	Editor access. Like Google Docs for code.

Class 1 — Data Cleaning

> **Scenario reminder:** Linda drops a 600,000-row CSV. You sample 100,000. The date column is text. Some workers wrote nothing in the advice field. Your job today: clean and shape.

Your goal

Sample, fix dates, deal with missing text fields.

Inputs

- `data/glassdoor_reviews.csv`

Outputs

- `reviews_step1.parquet` saved in Drive
- 3+ exploratory charts
- 1 chart for Linda
- Notes in markdown

Phase A — Explore the data first (15 min)

Exploratory chart 1 — Distribution of `overall_rating`

- **Question:** "Most workers rate around 3.5. Is it normal-shaped or skewed?"
- **HINTS:** Histogram with `bins=5` (ratings are 1-5).
- **What you learn:** Slight skew toward 4-5.

Exploratory chart 2 — `recommend` value counts

- **HINTS:** `df['recommend'].value_counts(dropna=False).plot.bar()`.
- **What you learn:** ~65% Yes, ~25% No, ~10% empty (missing).

Exploratory chart 3 — Missing values per column

- **HINTS:**
  - `df.isna().sum().sort_values(ascending=False).plot.barh()`.
- **What you learn:** `advice_to_mgmt` is the most-missing column (~30%).

Phase B — Clean the reviews table (45 min)

Step 1 — Load and sample

- **HINTS:**
  - `pd.read_csv('data/glassdoor_reviews.csv')`.
  - `.sample(100_000, random_state=42)`.
- **EXPECTED:** 100,000 rows.

Step 2 — Look at the DataFrame

- **WHAT:** Run `.info()`, `.head()`, `.shape`, `.dtypes`.
- **EXPECTED:** ~17 columns. `date_review` is `object` (text, not datetime).

Step 3 — Convert `date_review` to datetime

- **HINTS:**
  - `pd.to_datetime(df['date_review'], errors='coerce')`.
- **WHY:** We need to compute review age.

Step 4 — Find missing values

- **WHAT:** `df.isna().sum()`.
- **EXPECTED:**
  - `advice_to_mgmt`: ~30,000 missing.
  - `recommend`: ~10,000 missing (these are our "blind" rows from Linda's problem).
  - Sub-ratings (`work_life_balance`, `culture_values`, etc.): a few thousand missing each.

Step 5 — Decide what to do with each missing column

For each column with missing values, decide: - **A** Drop the row? - **B** Fill with a default value? - **C** Add a flag column "was missing"?

Common choices: - `recommend`: This IS our target. Drop rows where it is missing? Or keep them as "unknown" for prediction later? - For Module 3: drop these rows so we have a clean training target. - `advice_to_mgmt`: Replace empty with the string "no advice given". Keep the row. - Sub-ratings: Impute the median.

WRITE DOWN your decisions in markdown.

Step 6 — Drop rows with missing `recommend`

- **HINTS:** `df = df.dropna(subset=['recommend']).copy()`.
- **EXPECTED:** ~90,000 rows remain.

Step 7 — Replace empty text fields with a placeholder

- **HINTS:**
  - `df['advice_to_mgmt'] = df['advice_to_mgmt'].fillna('no advice given')`.
  - `df['pros'] = df['pros'].fillna('')`.
  - Same for `cons`, `headline`.

Step 8 — Save to Drive

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/glassdoor_lab/reviews_step1.parquet')`.

Phase C — Make ONE chart for Linda (15 min)

Linda's chart — "Where we are blind"

A bar chart showing 3 bars: - Recommend = Yes - Recommend = No - Recommend = Missing (= "where we are blind")

- **HINTS:**
  - `df['recommend'].value_counts(dropna=False).plot.bar()`.
  - Use `'v'` (vertical) bars.
  - Highlight the "missing" bar in red.
- **Title:** "10% of reviews leave recommend empty — that is 60,000 missing data points per year."
- **Takeaway for Linda:** "Our model fills these in. We deliver a complete report."

Common mistakes in Class 1

Mistake	What goes wrong
Forget `random_state=42` in sample	Different teammates have different samples. Hard to compare.
Drop the rows with missing `recommend` AFTER training	Leakage. Drop them in Class 1.
Forget `errors='coerce'` on `to_datetime`	One bad date crashes the code.

Self-check before Class 2

- ☐ You sampled 100,000 with `random_state=42`.
- ☐ `date_review` is datetime.
- ☐ Rows with missing `recommend` are dropped.
- ☐ Empty text columns are filled with a placeholder.
- ☐ 3+ exploratory charts.
- ☐ 1 chart for Linda.
- ☐ `reviews_step1.parquet` saved.

Class 2 — Encoding and Scaling

Scenario reminder: Linda says: "The `firm` column has 50,000 different company names. The sub-ratings are 1-5. Some workers are 'current employees', some are 'former'. Make these usable for the model."

Your goal

Encode `firm` (high-cardinality). One-hot the current/former status. Scale sub-ratings.

Inputs

- `reviews_step1.parquet`

Outputs

- `reviews_step2.parquet` in Drive

Phase A — Explore the data first

Exploratory chart 1 — Top 20 firms by review count

- **HINTS:** `value_counts().head(20).plot.bar()`.
- **What you learn:** Amazon, Google, Walmart dominate. Most firms have few reviews.

Exploratory chart 2 — Distribution of `overall_rating`

- **HINTS:** Histogram.

Exploratory chart 3 — Sub-ratings missing counts

- **HINTS:** For each sub-rating (`work_life_balance`, `culture_values`, `career_opp`, `comp_benefits`, `senior_mgmt`), count `.isna()`.

Exploratory chart 4 — `current` vs `former` employee distribution

- **HINTS:** `value_counts()`.
- **What you learn:** Maybe 70% current, 30% former.

Phase B — Encode and scale

Step 1 — Target-encode `firm`

- **WHAT:** 50,000 unique firms. One-hot is impossible. Target-encode = replace each firm name with the mean `recommend` rate for that firm.
- **HINTS:**
  - Compute mean of `recommend` per firm.
  - WARNING: only on train data in Class 4. For EDA now: `df.groupby('firm')['recommend'].transform('mean'`
- **WHY:** Reduces 50,000 categories to 1 number per row.

Step 2 — One-hot encode `current`

- **WHAT:** This column has values “Current Employee” or “Former Employee”.
- **HINTS:**
  - `df['is_current_employee'] = (df['current'] == 'Current Employee').astype(int)`.

Step 3 — One-hot encode top-20 `job_title` values

- **WHAT:** Many job titles. Keep top 20 + "Other".
- **HINTS:**
  - Find top 20: `df['job_title'].value_counts().head(20).index`.
  - For each: `df[f'job_{title}'] = (df['job_title'] == title).astype(int)`.

Step 4 — Impute sub-ratings with median

- **WHAT:** `work_life_balance`, `culture_values`, etc., have some missing values.
- **HINTS:**
  - For each sub-rating, compute median on training data.
  - Fill missing with median.
- **WARNING:** Only learn median from TRAIN. Apply same value to TEST.

Step 5 — Save

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/glassdoor_lab/reviews_step2.parquet')`.

Phase C — Make ONE chart for Linda

Linda's chart — "Top 20 firms by recommend rate"

A horizontal bar chart of the top 20 firms (with at least 100 reviews) by mean `recommend`.

- **HINTS:**
  - Filter to firms with >100 reviews.
  - GroupBy firm, mean recommend, sort, head 20.
- **Title:** "Top 20 employers by recommend rate."
- **Takeaway for Linda:** "These are our 'best employer' candidates. Sell them an 'Employer of Choice' badge."

Common mistakes in Class 2

Mistake	What goes wrong
Target-encode `firm` on FULL data	Leakage.
One-hot encode `firm` directly	50,000 new columns. Crash.
Impute sub-ratings BEFORE train/test split	Leakage.

Self-check before Class 3

- ☐ `firm` is target-encoded.
- ☐ `is_current_employee` exists.
- ☐ Top-20 job titles are one-hot.
- ☐ Sub-ratings have no missing values.
- ☐ 3+ exploratory charts.
- ☐ 1 chart for Linda.
- ☐ `reviews_step2.parquet` saved.

Class 3 — Feature Engineering

> **Scenario reminder:** Linda says: “Now make me the SMART columns. Length of the pros. Length of the cons. Does the worker mention ‘layoffs’ or ‘great pay’? These are the signals.”

Your goal

Engineer signals from the 4 text columns: `headline`, `pros`, `cons`, `advice_to_mgmt`.

Inputs

- `reviews_step2.parquet`

Outputs

- `reviews_step3.parquet` in Drive

Phase A — Explore the data first

Exploratory chart 1 — `pros_length` vs `recommend`

- After Step 1 below.
- **HINTS:** Bin `pros_length` into 5 groups, mean recommend, bar chart.
- **What you learn:** Workers who write LONG pros are more likely to recommend.

Exploratory chart 2 — `cons_length` vs `recommend`

- **HINTS:** Same pattern.
- **What you learn:** Workers who write LONG cons are LESS likely to recommend.

Exploratory chart 3 — `mentions_layoffs` rate by recommend

- After Step 3 below.
- **What you learn:** Mentions of “layoffs”, “fired” strongly predict NO recommend.

Phase B — Engineer the features

Step 1 — Length features

For each of the 4 text columns (`headline`, `pros`, `cons`, `advice_to_mgmt`):

New column	How to make it
`headline_length`	`df['headline'].fillna('').str.len()`
`headline_n_words`	`df['headline'].fillna('').str.split().apply(len)`
`pros_length`	`df['pros'].fillna('').str.len()`
`cons_length`	`df['cons'].fillna('').str.len()`
`advice_length`	`df['advice_to_mgmt'].fillna('').str.len()`

Step 2 — Pros / Cons ratio

- **WHAT:** A worker who writes MUCH MORE in pros than cons is positive.
- **HINTS:** `df['pros_to_cons_ratio'] = (df['pros_length'] + 1) / (df['cons_length'] + 1)`.

Step 3 — Keyword flags

For each of these keywords (case-insensitive), make a binary column:

Keyword group	Search words	New column
Compensation	“pay”, “salary”, “money”, “bonus”	`mentions_money`
Growth	“promotion”, “growth”, “career”, “learn”	`mentions_growth`
Layoffs	“layoff”, “fired”, “let go”	`mentions_layoffs`
Management	“manager”, “boss”, “leadership”	`mentions_management`

- **HINTS:**
  - Combine all 4 text columns into one big text string per row.
  - `df['mentions_money'] = combined_text.str.contains('pay|salary|money|bonus', case=False, regex=True).astype(int)`.
  - Same for other keywords.

Step 4 — Date features

New column	How to make it
`review_year`	`df['date_review'].dt.year`
`review_month`	`.dt.month`
`review_age_days`	`(pd.Timestamp.now() - df['date_review']).dt.days`

Step 5 — Save

- **HINTS:** `df.to_parquet('/content/drive/MyDrive/glassdoor_lab/reviews_step3.parquet')`.

Phase C — Make ONE chart for Linda

Linda’s chart — “Mentions of ‘layoffs’ predict NO recommend”

A bar chart showing the recommend rate for: reviews with “layoffs” mentioned vs without.

- **HINTS:**
  - GroupBy `mentions_layoffs`, mean recommend.
  - Bar chart.
- **Title:** “Reviews mentioning ‘layoffs’ have 35% recommend rate vs 70% baseline.”
- **Takeaway for Linda:** “Layoffs are a strong negative signal. Companies in mass-layoff news cycles should expect a big recommend drop.”

Common mistakes in Class 3

Mistake	What goes wrong
Compute keyword flags on EACH text column separately	You miss reviews where the word is in cons but not pros. Combine first.
Forget `.fillna('')` before `.str.len()`	Crashes on NaN cells.
Use simple substring search like `'pay' in text`	Pandas-specific syntax. Use `.str.contains()`.

Self-check before Class 4

- ☐ Length features for all 4 text columns.
- ☐ `pros_to_cons_ratio` exists.
- ☐ At least 4 keyword-flag columns exist.
- ☐ Date features exist.
- ☐ 3+ exploratory charts.
- ☐ 1 chart for Linda.
- ☐ `reviews_step3.parquet` saved.

Class 4 — Feature Selection

> **Scenario reminder:** Linda says: “You have ~30 columns. Pick the best 12-15.”

Your goal

Pick the best 12-15 columns.

Inputs

- `reviews_step3.parquet`

Outputs

- `reviews_step4.parquet` in Drive
- A short markdown report

Phase A — Explore the data first

Exploratory chart 1 — Correlation heatmap

- **HINTS:** `sns.heatmap(df[numeric_cols].corr(), annot=True, fmt='.2f')`.

Exploratory chart 2 — Mutual information bars

- After Step 4 below.

Exploratory chart 3 — Random Forest importance bars

- After Step 5 below.

Phase B — Select features

Step 1 — Split into train and test FIRST

- **HINTS:** `train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)`.

Step 2 — Variance threshold

- **HINTS:** `VarianceThreshold(threshold=0.01)`.

Step 3 — Correlation pruning

- **HINTS:** Drop pairs with |corr| > 0.9. Common candidates: `pros_length` and `pros_n_words`.

Step 4 — Mutual information

- **HINTS:** `mutual_info_classif(X_train[numeric_cols], y_train)`.

Step 5 — Random Forest importance

- **HINTS:** `RandomForestClassifier(n_estimators=50, max_depth=8, n_jobs=-1)`.

Step 6 — Pick the final 12-15 columns

- **WRITE DOWN** WHY for each.

Step 7 — Save

- Keep only selected + `recommend`. Save as `reviews_step4.parquet`.

Phase C — Make ONE chart for Linda

Linda's chart — "Top 10 predictors of NOT recommend"

A horizontal bar chart of top 10 features.

- **Title:** "Top 10 signals of 'would not recommend'."
- **Takeaway:** "Mentions of layoffs, low compensation rating, and low senior management rating are the strongest. Add an early-warning trigger when reviews mention these."

Common mistakes in Class 4

Mistake	What goes wrong
Compute mutual info on FULL data	Leakage.
Keep both pros_length and pros_n_words	Highly correlated. Pick one.

Self-check before Class 5

- ☐ Train/test split done FIRST.
- ☐ 12-15 columns + `recommend`.
- ☐ WHY for each column written.
- ☐ 3+ exploratory charts.
- ☐ 1 chart for Linda.
- ☐ `reviews_step4.parquet` saved.

Class 5 — Pipelines

Scenario reminder: Linda says: "Your code is in 4 notebooks. When a new review arrives, will you copy 4 notebooks to the server? We need ONE Pipeline."

Your goal

Build ONE Pipeline. Special challenge: 4 text columns need 4 separate TF-IDF vectorizers.

Inputs

- `reviews_step4.parquet`

Outputs

- `glassdoor_pipeline.joblib` in Drive

Phase A — Explore the data first

Exploratory chart 1 — Confusion matrix

- After training.

Exploratory chart 2 — ROC curve

- After training.

Phase B — Build the pipeline

Step 1 — Decide which columns are numeric, categorical, and text

- **EXAMPLE:**
  - numeric: `overall_rating`, `work_life_balance`, `culture_values`, `career_opp`, `comp_benefits`, `senior_mgmt`, `headline_length`, `pros_length`, `cons_length`, `advice_length`, `pros_to_cons_ratio`, `mentions_money`, `mentions_growth`, `mentions_layoffs`.
  - categorical: `is_current_employee`, `location_state`, top-20 `job_title` dummies.
  - text: `pros`, `cons`, `advice_to_mgmt`. (4 columns total — `headline` if you want).

Step 2 — Numeric mini-pipeline

- **HINTS:**
  - Skeleton:
    ```
    numeric_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='___')),
        ('scaler',  StandardScaler()),
    ])
    ```
  - Fill in: `'median'`.

Step 3 — Categorical mini-pipeline

- **HINTS:**
  - Skeleton:
    ```
    categorical_transformer = Pipeline(steps=[
        ('imputer', SimpleImputer(strategy='___')),
        ('onehot',  OneHotEncoder(handle_unknown='___', sparse_output=___)),
    ])
    ```
  - Fill in: `'most_frequent'`, `'ignore'`, `False`.

Step 4 — Multiple TF-IDF mini-pipelines (SPECIAL CHALLENGE)

- **WHAT:** You have 3-4 text columns. Each needs its OWN TfidfVectorizer.
- **HINTS:**
  - `from sklearn.feature_extraction.text import TfidfVectorizer`.
  - Skeleton:

```
pros_vec = TfidfVectorizer(max_features=___, ngram_range=(1, 2))
cons_vec = TfidfVectorizer(max_features=___, ngram_range=(1, 2))
advice_vec = TfidfVectorizer(max_features=___, ngram_range=(1, 2))
```

  - Fill in `max_features`: 500 each is enough for this dataset.

Step 5 — Combine everything in a ColumnTransformer

- **HINTS:**
  - `ColumnTransformer` takes a list of (name, transformer, column-or-columns).
  - For numeric: list of columns.
  - For categorical: list of columns.
  - For TF-IDF: a SINGLE column name as a string (not a list). Each TfidfVectorizer takes one column.

Step 6 — Add the model

- **HINTS:**
  - `LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42)`.
- **WHY `class_weight='balanced'`?** Slight imbalance (65/35).

Step 7 — Train and evaluate

- **HINTS:** `full_pipeline.fit(X_train, y_train)`. Then `classification_report`.
- **EXPECTED:** F1 around 0.75-0.85.

Step 8 — Save

- **HINTS:**
  - `joblib.dump(full_pipeline, '/content/drive/MyDrive/glassdoor_lab/glassdoor_pipeline.joblib`

Phase C — Make ONE chart for Linda

Linda's chart — "Our model fills 90% of the missing recommend fields correctly"

A simple bar showing: actual rate, predicted rate, and missing-filled rate.

- **HINTS:**
  - Show 3 bars: "Actual Yes rate", "Predicted Yes rate", "F1 on Yes class".
- **Title:** "Model performance — F1 of 0.82 means we fill missing recommend fields correctly 82% of the time."
- **Takeaway:** "We can deliver complete reports to HR teams now."

Common mistakes in Class 5

Mistake	What goes wrong
Pass a list of columns to TfidfVectorizer	TfidfVectorizer takes ONE text column at a time.
Forget `sparse_output=False`	OneHotEncoder returns sparse matrix.
Use max_features too high (e.g., 50,000)	Pipeline becomes huge and slow.

Self-check before Class 6

- ☐ Pipeline has preprocessor + model.
- ☐ Numeric, categorical, AND multiple TF-IDF transformers.
- ☐ F1 above 0.75.
- ☐ Confusion matrix + ROC curve charts.
- ☐ `glassdoor_pipeline.joblib` saved.

Class 6 — End-to-End Lab (THE BIG ONE)

> **Scenario reminder:** Linda is in the meeting room. 90 minutes. Deliver the clean dataset.

Your goal

Take the raw CSV. Produce ONE final `.parquet` file with the exact 21-column schema. Plus a 1-page findings report.

Time

90 minutes of focused work.

Outputs

- `glassdoor_reviews_clean.parquet` (~90,000 rows × 21 columns) in Drive
- `findings.md` (~1 page)
- Your Colab notebook

Phase A — Explore the data first (10 min)

Reuse 3 of your best charts: 1. `recommend` distribution (with the missing slice). 2. Layoffs mention vs recommend rate. 3. Top 10 most important features.

Phase B — Build the final dataset (70 min)

Required output schema

#	Column	Type	Source
1	`review_id`	int	raw
2	`firm_target_encoded`	float	engineered (train-only)
3	`is_current_employee`	int (0/1)	engineered
4	`location_state`	string	parsed from location
5	`overall_rating`	int 1-5	raw
6	`work_life_balance`	int 1-5	raw + median imputed
7	`culture_values`	int 1-5	raw + median imputed
8	`career_opp`	int 1-5	raw + median imputed
9	`comp_benefits`	int 1-5	raw + median imputed
10	`senior_mgmt`	int 1-5	raw + median imputed
11	`headline_length`	int	engineered
12	`pros_length`	int	engineered
13	`cons_length`	int	engineered

#	Column	Type	Source
14	`advice_length`	int	engineered
15	`pros_to_cons_ratio`	float	engineered
16	`mentions_money`	int (0/1)	engineered
17	`mentions_growth`	int (0/1)	engineered
18	`mentions_layoffs`	int (0/1)	engineered
19	`recommend`	int (0/1)	TARGET (mapped Yes/No -> 1/0)
20	`pros`	string	raw (kept for Module 7)
21	`cons`	string	raw (kept for Module 7)

Lab phases (timed)

Phase	Time	What you do
1. Load + sample	10 min	Load CSV, sample 100k.
2. Clean	15 min	Dates, drop missing recommend, fill empty text.
3. Encode	15 min	Target-encode firm, one-hot current/job, impute sub-ratings.
4. Engineer	20 min	Lengths, ratio, keyword flags, dates.
5. Validate + save	10 min	Check 21 columns + dtypes. Save `.parquet`.
6. Findings	10 min	Write `findings.md`.

Total: 90 minutes.

Phase C — Findings report for Linda (10 min)

Write `findings.md` with these sections:

Final numbers

- Final row count: ____
- Recommend rate: ____% (should be ~65-70%)

Top 3 insights

1. _____
2. _____
3. _____

One question to investigate in Module 4

- _____

One chart that summarizes everything

Embed your most important chart.

Grading rubric (100 points)

Item	Points
All 21 columns exist with right names and dtypes	25
Cleaning decisions in markdown	10
Pipeline reproducible (one command, raw CSV to `.parquet`)	15
Target encoding done on TRAIN ONLY	10
Multiple TF-IDF transformers in Class 5 pipeline	10
At least 3 exploratory + 1 explanatory chart per class	**15**
`findings.md` has insights	10
Code is clean	5

Submit

Upload to `module-3/class_6/submissions/<YourTeamName>/`: - `glassdoor_reviews_clean.parquet` - Your Colab notebook - `findings.md`

Tips for the whole module

1. **Do NOT copy-paste code from this guide.**
2. **Save your notebook to Drive often.**
3. **Save intermediate `.parquet` files to Drive.**
4. **Pair-program.** Switch every 20 minutes.
5. **Make a chart BEFORE and AFTER each cleaning step.**
6. **The 4-text-column challenge is unique to this scenario.** Plan how to handle multiple TF-IDF early.
7. **Ask the mentor early.** If stuck for 20 minutes, ask.

Good luck.